

Тестирање на апликацијата **FencingCoach**

Документација за систематско тестирање

Предмет:

Софтверски квалитет и тестирање

Изработил:

Петар Србиноски 231139

Линк до репозиториум: <https://github.com/PetarSrbinoski/FencingCoach>

Скопје, септември 2026

Документација за тестирање на FencingCoach

Документот ги опишува изборот на тестови, изолираната околина, извршените мерења и откриените дефекти. Прикажани се само проверки за кои постои зачуван резултат; тестови за перформанси, CI и реални надворешни провајдери не се прикажани како извршени.

Содржина

Поглавје	Содржина
1	Вовед, цел и опфат
2	Стратегија за избор на тестови и влезни блокови
3	Независен почетен сет и покриеност
4	Тестирање базирано на својства
5	Мутациско тестирање
6	PostgreSQL и HTTP API тестови
7	End-to-end тестови со Playwright
8	Откриени дефекти
9	Автоматизација и повторување
10	Резултати, ограничувања и заклучок

1 Вовед, цел и опфат

FencingCoach е апликација за еден корисник со Next.js интерфејс, FastAPI сервер, PostgreSQL база и интеграции со Garmin и јазичен модел. Функциите што се тестираат опфаќаат неделен распоред, извлекување метрики, пресметка на нутритивни цели, промена на типот на ден и евиденција на оброци. Целта е да се утврди дали независно изработени примери, генерирани својства и регресиски тестови разликуваат реални промени во однесувањето.

1.1 Граници на проверката

Постоечките backend/tests/ се регресиски сет на проектот, но не влегуваат во независните мерења. Во новиот сет функциите се тестираат директно, состојбата се проверува со мигрирана PostgreSQL база, а корисничките текови минуваат преку вистински HTTP сервер и Chromium. Надворешниот нутритивен агент и USDA повици имаат контролирани замени.

Ниво / техника	Што докажува	Алатка / граница
Примери и влезни блокови	Очекуван исход за валидни, невалидни и гранични вредности	pytest; јавни service функции
Својства	Инваријанти низ семејства на влезови	Hypothesis; чисти функции и изолирана БД
Мутации	Дали избраниот сет открива намерно изменет код	mutmut; само распоред и Garmin
Интеграција и API	Миграции, HTTP одговори и состојба	FastAPI + PostgreSQL 16
End-to-end	Видливо однесување низ вистински интерфејс	Playwright + Chromium

2 Стратегија за избор на тестови и влезни блокови

За почетниот сет е применето функционално партиционирање: секој договор е поделен на блокови со различно очекувано однесување, а потоа се избрани претставници и соседни вредности кај границите. Ова не е формално Base Choice Coverage за еден метод; методот е сличен по систематскиот избор на блокови, но нема такво тврдење за покриеност.

2.1 Карактеристики и блокови

Област	Основен валиден блок	Варијанти и граници
Неделен распоред	Седум препознаени вредности, понеделник до недела	0, 6 и 8 записи; непознат или празен токен; празни места и големи букви
Garmin скалар	Присутна вредност во дозволен опсег	Отсутна, null, ненумеричка, под/на самата/над границата; HRV = 0
Garmin приоритет	Вредност на првата поддржана патека	Прва патека недостасува; резервна патека; присутна, но невалидна прва вредност
Нутритивна цел	Валиден профил, фаза и тип на ден	Нема профил -> 89 kg; 60/80 kg; различна фаза и тип на ден
Историја на калории	Пет употребливи дена во претходните 14	Четири дена; ден -14, ден -15 и денес; невалиден статус
Рачен тип на ден	Запис за конкретниот датум	Замена, бришење, автоматски избор и соседен датум

2.2 Изведени случаи и очекувања

Случај	Избран влез	Очекуван резултат
S1	Седум валидни типови со мешана големина на букви	Нормализација без промена на редоследот
S2	Шест или осум типови; непознат токен	Одбивање пред употреба на распоредот
G1	HRV 0, 1 и 300	Нулата е недоволена; двете граници се валидни
G2	Нема прва HRV вредност, постои неделен просек	Резервната патека дава употреблива вредност
N1	Четири наспроти пет употребливи денови	Формула наспроти измерена основа за одржување
N2	Запис на ден -14, -15 и денес	Само претходните 14 дена учествуваат
D1	Рачна промена на еден датум	Промената е трајна и не го менува соседниот датум

3 Независен почетен сет и покриеност

Почетниот сет е замрзнат во локалниот commit d24a34d, пред додавање на својства и мутациски регресији. Трите датотеки во `testing/baseline/` ги проверуваат распоредот, Garmin извлекувањето и нутритивните цели. Нутритивните случаи работат со празна мигрирана база и синтетички спортист. Очекуваните исходи се запишани пред извршување на мутациите.

Мерен модул	Покриени / вкупно гранки	Примери што придонесуваат
Распоред	4 / 4	Должина, валидност и редослед
Garmin извлекување	22 / 32	Статуси, граници, патеки и payload
Нутритивни цели	17 / 24	Историја, профил и тип на ден
Вкупно	43 / 60 (71,7%)	24 успешно поминати pytest случаи

4 Тестирање базирано на својства

Hypothesis генерира семејства на поддржани влезови, а тестовите проверуваат правила што треба да важат за секој пример. Почетните датотеки не се менувани. Додадени се 13 функции: четири за распоред, шест за Garmin и три со PostgreSQL за нутритивните цели. Поставката `derandomize=True` ја прави селекцијата повторлива; бројот на генерирани примери е различен од бројот на `pytest` функции.

Област	Генератор / инваријанта	Поставени примери
Распоред	Валидни низи, нормализација, редослед и одбивање на невалидни токени	4 × 100
Garmin	Скаларни граници; речници со изоставени и <code>null</code> полиња; ограничени листи за батерија, <code>readiness</code> и <code>VO₂max</code>	6 × 100
Нутритивни цели	Тежина и тип на ден, енергетска согласност, изолирани датуми и историја	20 + 15 + 15
Вкупно поставени	13 property функции	1 050

За Garmin, секоја метрика со статус `ok` мора да има конечна вредност во документираниот опсег; другите статуси немаат доверлива скаларна вредност. Генераторите вклучуваат дозволени нумерички стрингови и облици на листи што ги прифаќаат `extractors`. `NaN` и бесконечност се посебни `in-process robustness` случаи, а не JSON податоци. За нутритивните цели се проверува согласноста меѓу `kcal` и макронутриентите и правилата за профил, датум и одржување.

Секој генериран PostgreSQL пример повикува заштитено ресетирање и повторно внесување на синтетичките редови во сопственото тело. Само `function-scoped fixture` не би ја изолирала базата меѓу примерите што Hypothesis ги извршува во една `pytest` функција. Проширениот запис покажува 37/37 успешно поминати случаи и 53/60 гранки (88,3%) за истиот измерен `production` код. Нема најдена грешка при генерирањето, па нема измислен минимизиран контрапример.

5 Мутациско тестирање

Mutmut 3.8.0 создава изменети верзии на кодот и ги извршува избраните тестови. За споредливост се мутираат само `schedule.py` и `garmin_extract.py`; нутритивните, `API`, `browser` и постоечките регресиски тестови не учествуваат во резултатот. Трите фази користат свежи работни простори, ист `hash` на двата `production` фајла и исти 466 `mutant ID`. Пред мерењето биле проверени чистото извршување и способноста на инструментот да убие намерно избран `mutant`.

Фаза	Избрани тестови	Убиени	Преживеани	Резултат
A	Почетни примери за распоред и Garmin	248	218	53,2%
B	A + својства за тие два модула	265	201	56,9%
C	B + три Garmin регреси	287	179	61,6%

Нема `mutant` без тест, `timeout`, грешка или прескокнат `mutant`. Резултатот е убиени / (убиени + преживеани + без тест) и нема корекција за еквивалентност. Во B, генерираните облици откриваат уште 17 промени, главно во калории, `body battery`, `training readiness` и `VO2max`. Во C, три случаи откриваат дополнителни 22: предност на вгнездена `sleep` патека, предност на директна `body-battery` вредност и задржување на дијагностички `payload`.

На пример, `mutant` што ја отстранува патеката `dailySleepDTO.sleepTimeSeconds` менува резултат од 2 на 1 час кога таа содржи 7 200 секунди, а резервната патека 3 600. Регресискиот тест ја прави разликата видлива. Друг `mutant` менува само формулација на детал за празен `status payload`; не е додаден кршлив тест за буквален текст без договор за него. Сите преживеани остануваат во именителот. Целосната анализа и зачуваните А, В и С резултати го ограничуваат толкувањето на двете мутирани датотеки.

6 PostgreSQL и HTTP API тестови

Посебен Docker Compose проект создава празна PostgreSQL 16 база, ги применува Alembic миграциите и стартува вистински FastAPI сервер. Тестот повикува HTTP endpoint, а потоа ја проверува и зачуваната состојба. Секој случај ги ресетира само синтетичките редови во тест-базата; заштитниот механизам ја проверува дестинацијата пред празнењето. Асинхроното генерирање оброк се набљудува со ограничено polling додека job не стигне во терминална состојба.

Сценарио	Главни проверки
Почетен smoke	Здравје на БД, денешни цели, завршен синтетички estimate job, без зачуван оброк
Рачен тип на ден	Заменува ист датум, одбива невалиден тип, не менува соседен датум, брише во Auto
Оброк	Estimate не логира; прегледаните вредности се менуваат, логираат еднаш, се сумираат и бришат
Неуспех и закрепнување	Контролиран estimate error е терминален; непознат ID дава 404; следна процена успева
Workout override	Замена, читање, валидација на sets/ reps/load/RPE, независен датум и бришење

Сите пет API/database случаи поминаа. Тест-околината го заменува само надворешниот нутритивен агент и USDA збогатувањето со предвидливи резултати; routing, background job, business logic и PostgreSQL се реални. JUnit и метаподатоците го идентификуваат извршувањето.

7 End-to-end тестови со Playwright

Playwright ја отвора production-изградбата на Next.js во Chromium и работи преку видливите контроли. Секој случај добива нов browser context, нова синтетичка состојба во БД и фиксирано пладне UTC за истиот ден што го користи backend. Селекторите се засновани на улоги и достапни имиња.

Бр.	Корисничко сценарио	Потврден исход
1	Страницата за исхрана се отвора и бара estimate	Вистински цели; контролирана процена стигнува до преглед
2	Корисник менува тип на ден и повторно вчитува	Рачниот избор и целите опстојуваат; Auto го враќа автоматскиот избор
3	Корисник прегледува, менува и зачувува оброк	Еден оброк и точни суми по reload; бришењето опстојува
4	Estimate паѓа, а потоа корисник се обидува повторно	Копчето се ослободува; нема оброк од грешката; нов преглед успева

Сите четири Chromium случаи поминаа без retry. Browser тестот за оброк ги проверува и прикажаните суми по повторно вчитување и HTTP агрегатите, наместо да се потпира само на појавување на текстот на оброкот. Playwright JSON е зачуван одделно од API JUnit.

8 Автоматизација и повторување

Извршувањето е преку локални bash скрипти, не преку CI. Скриптите создаваат единствен Compose проект и волумен, применуваат миграции, собираат JUnit/coverage/Playwright докази и ја отстрануваат само својата тест-околина. Изворните и тест-датотеките се идентификуваат со hash; mutation фазите зачувуваат конфигурација, mutant ID и генериран код. Главниот .env не се чита.

```
uv sync --frozen
bash testing/scripts/run_backend_tests.sh baseline
bash testing/scripts/run_backend_tests.sh expanded
bash testing/scripts/run_e2e.sh
bash testing/scripts/run_mutation_stage.sh A
bash testing/scripts/run_mutation_stage.sh B
bash testing/scripts/run_mutation_stage.sh C
```

Пред browser извршување се потребни npm ci и npm playwright install chromium во frontend/, како и Docker Compose. Командите за mutation се извршуваат последователно за споредливо времетраење. Тестирачкиот прирачник ги наведува портите и променливите за околината. Репозиториските quality gates се одделни од независните мерења.

10 Резултати, ограничувања и заклучок

Проверка	Потврден локален резултат
Независен почетен сет	24/24 успешни; 43/60 гранки во избраните service датотеки
Проширен сет со својства	37/37 успешни; 1 050 поставени примери; 53/60 гранки
Мутации	Исти 466 mutants; A 248, B 265, C 287 убиени
PostgreSQL + HTTP API	5/5 успешни
Playwright + Chromium	4/4 успешни, без retry
Постоечки репозиториски тестови	248 успешни; четири deprecation предупредувања
Ruff, mypy, TypeScript, frontend build	Успешни

Овие резултати даваат проверлива доверба за дефинираните service правила, трајноста на податоците и трите главни кориснички текови. Не следува заклучок за перформанси, безбедност, реален Garmin/LLM провајдер, медицинска точност или целокупна mutation покриеност на апликацијата.